

Kill Rate Is Not Enough: Selection Inflation and Candidate-Validity Regression in LLM-Based Mutation Test Generation

Abstract—Background. Large language models are increasingly used to generate unit tests, and mutation kill rate is a common way to score them. Kill rate alone rewards any candidate that distinguishes a mutant from its reference, including a test that fails universally. **Aims.** We measure how far a development-panel gain transfers to a locked split, whether kill rate and candidate validity move together under supervised fine-tuning, and whether a full-scale rehearsal on a permitted split can establish that an evaluation pipeline computes what it claims. **Method.** Four arms of a 1.5B-parameter code model were evaluated under one frozen generation protocol with eight candidates per target at seed 42, on a selection panel of 542 function targets. One checkpoint was then evaluated against the untrained base on a locked split of 757 targets under a promotion rule committed before either arm ran. A receipt-bound rehearsal re-executed the entire measurement path at full scale on the permitted panel. **Results.** Kill@8 rose from 0.605166 to 0.695572 on the selection panel. On the locked split the paired difference was +3.4346 points (450/757 $\rightarrow$ 476/757; +114/−88; net +26) with exact two-sided McNemar $p = 0.0783$ against a predeclared $p < 0.01$, so the base model was retained. Reference validity fell 11.8726 points over the same comparison while parse and execution validity did not regress. The rehearsal reproduced a prior Kill@8 of 0.605166 exactly across 4,336 candidates with complete raw-output integrity. **Conclusions.** A selection-panel gain is not evidence of generalization; a kill-rate-only report can hide a validity regression moving the other way; and executing the whole measurement path on permitted data at full scale is a practical precondition for trusting any irreversible measurement.

Index Terms—mutation testing, test generation, large language models, empirical evaluation, candidate validity, reproducibility

I. INTRODUCTION

Automated test generation is being rebuilt around large language models, and the field needs scoring functions that survive contact with a learned optimizer. Mutation kill rate is an attractive choice: it is objective, executable, and independent of coverage heuristics. But Kill@k alone can be misleading when candidate and reference validity are not reported alongside it. A test that fails against *every* implementation still distinguishes a mutant from its reference, so a kill-rate-only evaluation cannot separate a discriminating test from a broken one.

This paper reports an empirical study of that pressure, and of the evaluation machinery required to observe it. We fine-tuned a 1.5B-parameter code model to generate mutant-killing unit tests, evaluated four arms on a panel used for checkpoint selection, and then evaluated the best checkpoint against the

untrained base on a locked split that had not been used to select a checkpoint, tune a prompt, or set a threshold, under a decision rule fixed in advance.

The selection panel showed a 9.04 point Kill@8 gain. The locked split showed 3.4346 points, which did not meet the predeclared significance threshold; the untrained base model was retained. Over that same locked comparison, the fraction of candidates that pass against the *correct* implementation fell by 11.8726 points, while parse and execution validity did not regress. The tuned model wrote more mutant-killing tests that more often failed the reference.

We also report an evaluation-integrity failure with direct methodological consequences. A one-time final measurement was authorized and consumed without producing any measurement, because a record-admission rule in the final-run path ran before the scope filter that the established evaluator had always applied first. The defect was reproducible on permitted splits at no cost, and survived review because the component was unreachable until the authorization was spent. The safeguard we built in response — a receipt-bound, full-scale rehearsal on a permitted split — reproduced a previously measured value exactly, which is evidence about execution fidelity and nothing else.

Research questions and contributions

- **RQ1:** How much does a selection-panel gain overstate the locked result? (Section IV)
- **RQ2:** How do Kill@k and candidate validity co-vary under SFT? (Section V)
- **RQ3:** How do parsing semantics, raw-output retention and split isolation affect interpretability? (Section VI)
- **RQ4:** Can a receipt-bound full-scale rehearsal on a permitted split reproduce a prior measurement without touching a consumed split? (Section VII)

Our contributions are a quantified selection-inflation measurement with a predeclared decision rule, a kill-rate-versus-validity trade-off isolated from parse and execution effects, a rehearsal protocol that makes irreversible measurement safer, and a fully receipted replication package (Section X).

II. BACKGROUND AND RELATED WORK

a) *Mutation testing:* Mutation analysis injects small faults and asks whether a test suite detects them [1], [2]. Its use as a proxy for real fault detection has been examined directly,

with important caveats [3]–[5]. We use mutation kill rate as a scoring function, and this paper is partly about what happens when it becomes an optimization target.

b) Automated and LLM-based test generation: Search-based and feedback-directed tools set the baseline [6]–[8]. LLM-based generation is now an active area [9]–[11], with surveys covering the landscape [12], [13]. Reported gains are commonly expressed in coverage or pass rate; comparatively little attention is paid to whether a generated test is valid against the correct implementation.

c) Evaluation rigour in testing research: The fuzzing community has confronted this directly: Klees *et al.* demonstrated that many published fuzzing comparisons do not survive controlled re-evaluation [14], against widely deployed tools [15], [16] and, more recently, LLM-driven fuzzers [17], [18]. Statistical guidance for randomized algorithms in software engineering is well established [19], as is caution about thresholded significance [20], [21].

d) Leakage, contamination and reproducibility: Leakage inflates machine-learning results across disciplines [22]; contamination of code benchmarks is documented [23]–[26]; reproducibility has been studied broadly [27]–[29] and for LLM use in software engineering specifically [30].

III. STUDY DESIGN

A. Subject system and protocol

The generator is Qwen2.5-Coder-1.5B-Instruct at a pinned immutable revision. Every measurement in this paper runs under one named protocol, identified by digest and frozen before use: eight candidates per target, temperature 0.7, top- p 0.9, seed 42, whole-output candidate parsing, raw-output retention, completion limit 1024 tokens, sequence limit 3072 tokens, batch size two, no reranking, no feedback rounds. The four development arms ran under one recorded contract digest and one evaluation-scope digest, verified identical across arms.

B. Metrics

Kill@ k is the fraction of targets for which at least one of the first k candidates, in raw generation order, kills the mutant. *Parse validity* is the fraction of candidate slots yielding a usable test; *execution validity* the fraction running without infrastructure error; *reference validity* the fraction passing against the correct implementation. Reference validity is the metric that distinguishes a discriminating test from a broken one, and it is the one we find regresses.

C. Splits, roles and the evaluation workflow

Figure 1 shows the workflow and the role of each split. The development panel selected the checkpoints it scored. The locked split was not used to select a checkpoint, tune a prompt, or set a threshold; it determined the predeclared promotion decision, and it is the only generalization measurement here. A fourth split was reserved for a single final measurement; it was consumed without producing one, and no number in this paper derives from it.

TABLE I
MAIN RESULTS. THE TWO BLOCKS ARE NOT COMPARABLE QUANTITIES.

Panel	Arm	Kill@8	Fns	Wilson 95%
<i>Development — selection-only, $n = 542$</i>				
	base	0.605166	328	[0.5634, 0.6454]
	A@150	0.690037	374	[0.6499, 0.7275]
	A@431	0.695572	377	[0.6556, 0.7328]
	B@150	0.658672	357	[0.6178, 0.6973]
<i>Locked validation — not used for selection, $n = 757$</i>				
	base	0.594452	450	[0.5591, 0.6289]
	A@431	0.628798	476	[0.5938, 0.6625]
<i>Paired locked difference</i>				
	gained / lost / net			+114 / -88 / +26
	killed by both / neither			362 / 193
	Kill@8 delta			+3.4346 points
	McNemar exact two-sided p			0.0783
	predeclared requirement			$p < 0.01$ — not met
	decision			retain base model

D. Predeclared decision rule

Before either locked arm ran, a promotion rule was frozen in a hash-bound receipt. Criterion 1 required *all* of: Kill@8 gain $\geq +3.0$ points, net gain ≥ 15 functions, and exact two-sided McNemar $p < 0.01$. Further criteria bounded permitted regression in reference validity, parse and execution validity, and candidate diversity. Fixing the threshold beforehand is what makes the outcome reported in Section IV interpretable.

IV. RQ1: SELECTION INFLATION

Table I and Figure 2 give the answer. On the selection panel, Kill@8 rose from 0.605166 to 0.695572, a gain of 9.04 points. On the locked split the same checkpoint gave 0.628798 against 0.594452: +3.4346 points, roughly $2.6\times$ smaller.

The shrinkage is what selection bias predicts: the development panel chose the checkpoint it then scored. The locked figure is positive, and it is the only generalization estimate here.

It is equally important what the locked figure does not establish. With 202 discordant pairs (114 versus 88), an exact two-sided McNemar test [31], [32] yields $p = 0.0783$ against a required $p < 0.01$. The data are insufficient to conclude an effect at the standard fixed in advance. That is not the same as concluding there is no effect [20], [21], and we make no such claim. The question is undetermined at this sample size and cannot be resolved by re-measuring this split, which has now been used to decide.

V. RQ2: KILL RATE AGAINST CANDIDATE VALIDITY

Table II shows the trade-off. Reference validity fell from 0.452114 to 0.333388 on the locked comparison, a -11.8726 point change, and targets with *any* reference-valid candidate fell from 675 to 566. Over the same comparison parse validity rose 0.1486 points and execution validity rose 1.3870 points.

Three observations. First, the direction is consistent across all four development arms, so it is a property of this supervision family rather than of one checkpoint. Second, because

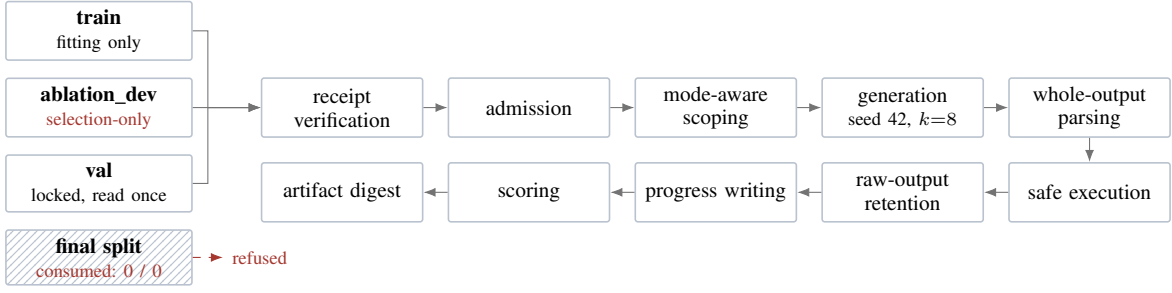


Fig. 1. Evaluation workflow and split roles. The rehearsal (Section VII) executes this entire chain on the permitted panel and is an operational check of execution fidelity, **not** model selection. For the consumed split we give neither size, membership, record counts nor any performance score; the two zeros shown record the historical fact that the authorized attempt generated zero candidates and zero reportable metrics.

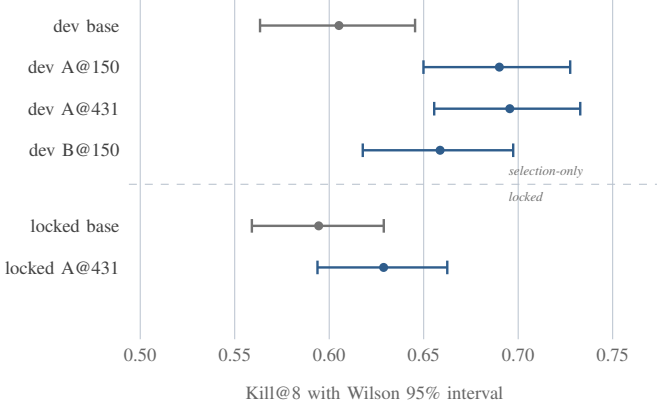


Fig. 2. Development and locked Kill@8 with Wilson 95% intervals. The locked pair (bottom two rows) is **positive but not significant** under the predeclared $p < 0.01$ rule; its intervals overlap. Development and locked rows measure different populations and must not be subtracted.

TABLE II
CANDIDATE VALIDITY. THE REGRESSION IS SPECIFIC TO REFERENCE VALIDITY; PARSE AND EXECUTION VALIDITY IMPROVED.

Panel	Arm	Parse	Exec.	Ref.	Fns ref-valid
<i>Development — selection-only</i>					
	base	0.983395	0.948570	0.565498	—
	A@150	0.997463	0.975092	0.458487	—
	A@431	0.998155	0.993081	0.455028	—
	B@150	0.998155	0.973478	0.448801	—
<i>Locked validation</i>					
	base	0.987285	0.928336	0.452114	675
	A@431	0.988771	0.942206	0.333388	566
	delta	+0.1486	+1.3870	-11.8726	-109

parse and execution validity *improved*, the regression is not explained by malformed or non-running code — the tuned model produces more usable tests that are more often wrong about the reference. Third, the structural explanation is available and testable: kill rate credits any candidate that separates mutant from reference, and a test that fails both separates neither, but a test that fails the reference while failing the mutant differently can still be credited. A validity-blind objective therefore admits solutions a practitioner would reject.

We report this as an observed association under one recipe.

We did not ablate the objective, and we do not claim the trade-off is unavoidable.

VI. RQ3: INTERPRETABILITY OF REPORTED NUMBERS

a) Parsing semantics are part of the measurement:

Candidate extraction supports taking the first assertion or the whole output. These score different artifacts. An earlier version of our final-run path never set the mode explicitly and would have inherited a legacy default, producing a plausible number under one parser while the frozen record named another. Parsing mode belongs in the frozen protocol beside the seed.

b) Raw-output retention makes results auditable:

Every candidate slot retains its raw model output and a digest. This permitted an independent recomputation of all 4,336 digests in the rehearsal, and it lets a disputed score be re-examined without regeneration. Pipelines that store only verdicts cannot separate a scoring bug from a model behaviour after the fact.

c) Split isolation must be structural:

Split roles are meaningful only if code cannot reach the wrong split. Our final-split guard gated on *authorization* — a fact that, once recorded, never expires — rather than on whether anything remained to measure. A permission check is not an exhaustion check, and the distinction cost us a split.

d) Leakage:

Where targets come from public benchmarks, a model may recall an assertion rather than derive one [22]–[24]. We classify assertion *form* separately from assertion *value* so that a gain cannot be credited to the wrong mechanism, and we exclude records of one execution mode from every rate because no validated evaluator exists for them.

VII. RQ4: FULL-SCALE REHEARSAL

A. The consumed measurement

A one-time final measurement was authorized and attempted. It reached `failed_after_authorization`: the single-use authorization was spent and the run raised while admitting records. It produced **zero candidates and zero reportable metrics**. Nothing was generated and nothing was scored. The split is consumed and may never be rerun; a split that has been opened is no longer held out regardless of what it produced.

The cause was a record-admission rule requiring a fixed field set to be non-empty on *every* record in a split, raising

TABLE III
OPERATIONAL REHEARSAL. EVIDENCE OF EXECUTION FIDELITY ONLY.

Quantity	Value
Requested records	594
Scored targets (function mode)	542
Excluded by mode	52
Admission errors	0
Candidates (542×8)	4,336
Generation batches $\times$ size	271×2
Raw outputs retained / digests	4,336 / 4,336
Missing / mismatched	0 / 0
Independently recomputed mismatches	0
Parse failures	72
Execution failures	223
Timeouts	43
Prompt-budget failures	0
Wall time / exit status	577.753 s / 0
Weights written	0
Kill@8	0.605166
Prior recorded value	0.605166 (exact match)

before returning any. Records of one execution mode legitimately carry some of those fields blank, and the established evaluator had always loaded the split and *then* scoped scoring by mode. The final path inverted that order and refused exactly the records the pipeline was always going to exclude. The rejection reproduces immediately on permitted splits, at no cost. A mode-only exemption would still have been wrong: on the locked split, three records of the *scored* mode also carry one such field blank and were measured.

B. Design and outcome of the rehearsal

We rebuilt admission to partition by execution mode *first* and apply field requirements only to records that will be scored, importing the mode semantics from the established evaluator rather than restating them. We then executed the entire chain of Figure 1 at full scale on the permitted panel, under a hash-bound receipt whose source bindings are verified before the model is loaded.

Table III reports the outcome. The decisive figure is that Kill@8 came out at 0.605166, identical to the previously recorded value for the same model on the same panel under the same evaluation-scope digest.

The scope of this result must be stated precisely. It is evidence that the rebuilt path computes the same number as the path that produced the historical value. It is *not* a model-performance result, *not* a generalization estimate, *not* a selection result, *not* a comparison against any baseline tool, and *not* a replacement for the lost measurement. The rehearsal receipt carries `final_test_measurement: false`, `eligible_for_model_selection: false` and `supports_performance_claim: false` as machine-checkable fields, and the runner refuses a receipt lacking them.

VIII. DISCUSSION

a) For practitioners scoring LLM test generators: Report a validity metric beside the kill rate. The regression we

measured is invisible to a kill-rate-only report and moves opposite to the headline, so a tool selected on kill rate alone may be selected against test quality.

b) For anyone using a held-out split once: Execute the full path on permitted data at full scale before the irreversible run. A smoke test on synthetic records proves the generator, not the pipeline. Make the one-time authorization two-phase so a post-open failure does not consume the split, and gate on exhaustion rather than permission.

c) On thresholds: A predeclared threshold is uncomfortable precisely when it binds. Our $p = 0.0783$ was close enough to 0.01 that any rule written afterwards would have been suspect. Fixing it in a hashed artifact before the run is what makes the retention decision reportable at all.

IX. THREATS TO VALIDITY

a) Construct: Kill@k credits any distinguishing candidate, including a universally failing test; we therefore report reference validity beside it. Mutants are an imperfect proxy for real faults [3], [4].

b) Internal: One locked criterion failed through a defect of ours: an adapter-specific digest was placed inside a contract required to be identical across arms, which no base/adaptor pair can satisfy. We record it as a failure rather than reinterpreting it; the decision does not rest on it because the significance criterion failed independently. All development arms ran under one contract and one scope digest, recorded identical, removing protocol drift as an explanation.

c) External: One model, one parameter scale, one corpus, one supervision family, one seed. We do not claim the shrinkage ratio or the validity trade-off transfers. Records of one execution mode are excluded from every rate, so no claim is made about that target class, and no native repository-level kill rate exists.

d) Statistical conclusion: A single paired test on 202 discordant pairs. We report an exact test and Wilson intervals [19], [33] and avoid converting a non-significant result into a null claim.

e) Reproducibility: The consumed split cannot be reproduced by anyone. Everything else is reproducible from committed receipts; the rehearsal is direct evidence, having reproduced a prior value exactly.

X. REPLICATION PACKAGE

An anonymized replication package accompanies this submission. Every numeric value in this paper is traceable to one of the following receipts by SHA-256. Paths are relative to the package root; no host, account or repository URL is included, in keeping with double-anonymous review.

The package includes: the frozen protocol definition and its digest; the promotion rule as committed before the locked run; per-arm metric receipts for both panels; the rehearsal receipt and execution receipt; per-figure data files, each carrying the digest of its source receipt in a header comment; and a command list that re-derives every table from the receipts alone.

TABLE IV
RECEIPTED ARTIFACTS. DIGESTS ARE TRUNCATED FOR SPACE; FULL
VALUES ARE IN THE PACKAGE MANIFEST.

Artifact	SHA-256 (prefix)
Frozen development evaluation receipt	062774028693cfcdb
Development selection receipt	0188324fe9eb2a51
Locked-validation preflight (rule frozen)	97be6855d41b1186
Locked-validation result receipt	f7cdf84df8e65c51
Rehearsal receipt (pre-run freeze)	bae9510218641b67
Rehearsal execution receipt (sanitised)	06aca918d2821a1f

Raw generated outputs are retained but excluded from the package: they comprise 4,336 model completions for the rehearsal alone. Their individual digests are published, so any retained output can be verified without redistribution. No material from the consumed split is included, and none exists.

XI. CONCLUSION

A 9.04 point development-panel gain corresponded to +3.4346 points on a locked split, which did not meet a predeclared significance threshold; the untrained base model was retained. Over the same comparison, candidate reference validity fell 11.8726 points while parse and execution validity improved — a trade-off a kill-rate-only evaluation cannot see. A receipt-bound full-scale rehearsal on a permitted split reproduced a prior measurement exactly, establishing execution fidelity for the pipeline without touching a split that had already been consumed by a failed one-time attempt.

The measurement practices that made these statements possible are cheap individually and only valuable together: freeze the rule, bind the sources by digest, retain raw outputs, keep split roles structural, and run the whole path on permitted data before anything irreversible depends on it.

REFERENCES

- [1] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
- [2] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, “Mutation testing advances: An analysis and survey,” in *Advances in Computers*. Elsevier, 2019, vol. 112, pp. 275–378.
- [3] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, “Are mutants a valid substitute for real faults in software testing?” in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2014, pp. 654–665.
- [4] M. Papadakis, D. Shin, S. Yoo, and D.-H. Bae, “Are mutation scores correlated with real fault detection? a large scale empirical study on the relationship between mutants and real faults,” in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, 2018, pp. 537–548.
- [5] R. Just, D. Jalali, and M. D. Ernst, “Defects4j: A database of existing faults to enable controlled testing studies for java programs,” in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2014, pp. 437–440.
- [6] G. Fraser and A. Arcuri, “EvoSuite: Automatic test suite generation for object-oriented software,” in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, 2011, pp. 416–419.
- [7] —, “Whole test suite generation,” *IEEE Transactions on Software Engineering*, vol. 39, no. 2, pp. 276–291, 2013.
- [8] C. Pacheco and M. D. Ernst, “Randoop: Feedback-directed random testing for java,” in *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA)*, 2007, pp. 815–816.
- [9] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, “CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models,” in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 919–931.
- [10] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.
- [11] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, “Unit test case generation with transformers and focal context,” arXiv preprint arXiv:2009.05617, 2020.
- [12] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software testing with large language models: Survey, landscape, and vision,” *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024.
- [13] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, 2024.
- [14] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, “Evaluating fuzz testing,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 2123–2138.
- [15] K. Serebryany, “Continuous fuzzing with libfuzzer and addresssanitizer,” in *IEEE Cybersecurity Development (SecDev)*, 2016, p. 157.
- [16] M. Böhme, V.-T. Pham, and A. Roychoudhury, “Coverage-based grey-box fuzzing as markov chain,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 1032–1043.
- [17] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, “Fuzz4all: Universal fuzzing with large language models,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024.
- [18] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, “Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, pp. 423–435.
- [19] A. Arcuri and L. Briand, “A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering,” *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.
- [20] R. L. Wasserstein and N. A. Lazar, “The ASA statement on p-values: Context, process, and purpose,” *The American Statistician*, vol. 70, no. 2, pp. 129–133, 2016.
- [21] V. Amrhein, S. Greenland, and B. McShane, “Scientists rise up against statistical significance,” *Nature*, vol. 567, no. 7748, pp. 305–307, 2019.
- [22] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023.
- [23] S. Yang, W.-L. Chiang, L. Zheng, J. E. Gonzalez, and I. Stoica, “Rethinking benchmark and contamination for language models with rephrased samples,” arXiv preprint arXiv:2311.04850, 2023.
- [24] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html
- [25] M. Chen, J. Tworek, H. Jun, Q. Yuan *et al.*, “Evaluating large language models trained on code,” arXiv preprint arXiv:2107.03374, 2021.
- [26] J. Austin, A. Odena, M. Nye, M. Bosma *et al.*, “Program synthesis with large language models,” arXiv preprint arXiv:2108.07732, 2021.
- [27] O. E. Gundersen and S. Kjensmo, “State of the art: Reproducibility in artificial intelligence,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.
- [28] M. Hutson, “Artificial intelligence faces reproducibility crisis,” *Science*, vol. 359, no. 6377, pp. 725–726, 2018.
- [29] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché Buc, E. Fox, and H. Larochelle, “Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program,” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021. [Online]. Available: <https://jmlr.org/papers/v22/20-303.html>
- [30] J. Sallou, T. Durieux, and A. Panichella, “Breaking the silence: The threats of using llms in software engineering,” in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2024, pp. 102–106.

- [31] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [32] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.
- [33] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.